VIBE CODING 方法论

SKILL 与 MCP SERVER 实战

LESSON 03 · 本节课 · 最终话

Skill × MCP 协同
如何提升 Vibe Coding

可靠闭环 = 故障下也不崩 =

纪律 + 手

DISCIPLINE + HANDS. 缺一个，闭环就断。

> “
>
> MCP 给**机器可判的真实反馈**，
> skill 给**必须用反馈的纪律**。

主讲人	**待定**
学期	**2026 秋季**
时长	**45 分钟 · 含实验**
前置	**你的 skill + devboard MCP**

课程地图
COURSE MAP

三节递进：方法论 → 能力 → 融合

01
写 Skill
方法论 · 闭环 SOP

02
写 MCP Server
能力 · agent 的手脚

03 本节
Skill × MCP 协同
融合 · 故障下不崩

课程地图：正 → 反 → 合

方法论 → 能力 → 融合
同一份作品，迭代了三节课

递进结构 · 本节收束全线

课次	主题	一句话定位	实验
01	**写 Skill**	把「写→查→验」固化成 agent 的 SOP	修复坏 skill（纯软件闭环）
02	**写 MCP Server**	把开发板 API 变成 agent 的手脚	补全 `tools` + 点灯
03	**Skill × MCP 协同** 本节	纪律 + 手 = 可靠闭环，故障下不崩	协同改造 + **故障注入挑战**

DEFINITION OF WORK

「保证它能 work」
两个定义本节同时落地

1 **换一个不知道内情的人，按 skill 也能复现闭环**
→ 验收：**跨组验收（本节 L2 · 20 分）**

2 **逆风局（故障注入）下行为可预期**
→ 验收：**混沌测试（本节 L2 · 三场景）**

今日议程

六个环节 · 最终话
页码 = 本 deck 页码

页码	环节	内容
P04	**开场 A/B 演示 · 0–6′**	同一个点灯任务跑三种配置 —— 对比**回合数与结果**
P05	**三课对照 · 定位**	L1 / L2 各没做成什么事 —— 本课**不装新零件，只焊**
P06–08	**协同本质与机制 · 6–16′**	感知—判断—行动循环 · **融合点契约** · 五模式 + 反模式
P09–10	**精读融合版 skill · 16–25′**	3 个 TODO 空位 · 引用 tool 的正确写法 · 诊断分支与停止条件
P11–14	**实验三讲解 · 25–28′**	材料 · 三层任务 · 混沌三场景 · rubric
P12 驻屏	**实验三** 动手	**L0 transcript 分析 → L1 协同改造 → L2 故障注入** · 28–42′
P15	**展示 + 三节课总结 · 42–45′**	「work」的完整定义 · 拓展作业

同一任务，三种 agent

三段**真实会话记录**（transcripts/）
任务：让板载 LED 以 64% 亮度点亮

CONFIG A · SKILL-ONLY

只有纪律，没有手

skill ✓ | ~~MCP ×~~

3 passed
软件侧全绿

✕ 真板 LED 根本没亮

闭环断在「验」

pytest 用的是 **mock**——验证对象是代码逻辑，不是板子状态。flash、读回、日志**一个都没发生**；想查板子只能 curl 连接失败。

CONFIG B · MCP-ONLY

只有手，没有纪律

~~skill ×~~ | MCP ✓ · 7 tools

47 轮相同调用
无限重试不收敛

✕ 拿「返回 OK」当成功

自证循环

写 GPIO**15** 读 GPIO**15**，永远一致、永远无新信息；get_logs / adc_read / pins.md 一次没用——LED 明明在 **GPIO2**。

CONFIG C · SYNERGY

纪律 + 手

skill ✓ | MCP ✓ · 7 tools

1 轮通过
零重试

✓ 交付附完整证据链

证据链 · EVIDENCE

pins.md 物理特性推出判定区间 **[2100, 2320]** → adc_read raw = **2214 / 2201** 两次命中 → get_logs 佐证 flash + pwm_set 事件。

—— **变量不是模型，是「纪律与手是否同时在场」——这节课把两者焊在一起**

三节课，各没做成一件事

不是三个知识点，是**同一条闭环的三次补焊**

本课不装新零件

LESSON 01 · 只有 SKILL

装了纪律

有纪律，没有手

装了什么

闭环 SOP：写→查→验 · 有限迭代 · 停止条件

验证停在哪

软件世界——ruff / pytest（mock 替身）

没做成的事

agent 摸不到板子：**全绿 ≠ 灯亮**，闭环断在「验」

当时 WORK =

换一组人，按 skill 能**复现软件闭环**

LESSON 02 · 只有 MCP

装了手和眼

有手，没有纪律

装了什么

devboard MCP · 7 个 tools（写 / 读回 / 日志 / 烧录）

验证停在哪

单次调用返回——OK / ERROR 文本

没做成的事

没人强制使用反馈：**拿 OK 当成功**，47 轮空转

当时 WORK =

工具返回值**机器可判**（但判什么没人管）

LESSON 03 · SKILL × MCP

把两者焊起来

纪律指挥手

装了什么

没有新零件——把第 2 课的手，写进第 1 课的验证段

验证停在哪

硬件在环——读回比对 + 光敏物理佐证 + 板端日志

做成的事

逆风局（混沌注入）下**行为可预期**：恢复或优雅认输

现在 WORK =

可交换复现 **+** 逆风局可预期，**两条同时落地**

区别一句话：L1 管「必须验」，L2 管「拿什么验」，L3 把两者写成同一条流程

Vibe Coding = 一个循环

不是「随缘生成」
是 agent 的**感知 – 判断 – 行动**循环

循环的三个环节

SENSE · JUDGE · ACT

上一课的 47 轮死循环，就是这个环**断了还在空转**。三个环节各有各的供给方：

1 行动 ACT

调用工具**改变世界**：gpio_write · pwm_set · firmware_flash

`MCP · 手`

2 感知 SENSE

读回世界的**真实反馈**：gpio_read · adc_read · get_logs

`MCP · 眼`

3 判断 JUDGE

用**机器可判标准**决定：区间命中？错误归类？继续 / 停止 / 升级给人？

`SKILL · 纪律`

闭环之所以「闭」

ACT → SENSE → JUDGE → ACT



✕ **缺 MCP = 判断拿不到真实反馈（软件自嗨） · 缺 skill = 反馈没人强制使用（47 轮空转）**
—— 缺一个，闭环就断。这正是 A/B 演示里两种失败的解剖学。

融合点不是代码，是一份契约

skill 与 MCP **互不看对方内部**
只凭「名字 + 返回文本」相认

SKILL.md — 纪律 · 一份文本

它管

- **何时调、按什么顺序**调工具
- 什么算**通过**（机器可判标准）
- 失败时**怎么办**（诊断分支）
- 何时**停下、升级给人**

✕ 不抄参数细节

1 编排缝

skill ──▶ mcp

穿过 = **工具名字 + 调用顺序**

2 反馈缝

mcp ──▶ skill

穿过 = **返回文本** OK/ERROR+数值

3 失败缝

mcp ──▶ skill

错误文本 → 诊断分支 → 重试/reset/升级

MCP server — 能力 · 一个进程

它管

- **怎么调**：docstring / schema 即说明书
- 参数**校验**与范围（越界返回 ERROR）
- **错误文本**格式（可读、可归类）
- 与板子 / 模拟器的**真实 I/O**

✕ 不知道流程

拿一步验证走完这份契约（synergy transcript 实测）

ONE STEP, END TO END

① SKILL 条款
「点亮后用 adc_read 读回，raw ∈ [2100,2320] 才算过」
只写名字 + 判定标准

→

② AGENT 编排
调用 adc_read(34)
参数含义查 docstring，不查 skill

→

③ MCP 返回
"OK: raw = 2214"
机器可判的文本，不是异常栈

→

④ SKILL 标准判
2214 ∈ 区间 → 过
越界 → 进诊断分支（P10）

协同五模式

每条都能在 `synergy transcript` 里指认
右栏两条反模式 = 协同的常见死法

1 skill 编排工具

验证段直接写调用序列：「`pwm_set` 点亮后，用 `adc_read` 读回比对，再 `get_logs` 佐证」——流程长在 skill 里，不靠 agent 临场发挥。

2 工具返回值 = 机器可判标准

`OK` / `ERROR` 前缀、数值区间写进成功条件：**raw ∈ [2100, 2320]**。**[0, 4095] 类永真式 = 0 分**——区间必须能从 `pins.md` 物理特性推导。

3 双重护栏

危险操作两层各挡一次：**skill 层**要求先向用户展示路径并获同意，**MCP 层** schema 再校验 `confirm` 参数——`firmware_flash` 无确认一律拒绝。

4 渐进披露

引脚表与光敏物理特性放 skill `references/` 与 MCP resources，agent **按需取用**——判定区间就是从这份披露里推出来的，不是拍脑袋。

5 失败升级路径

tool 错误文本 → 诊断分支 → **超阈值停止，报告给人**。整张分支表下一页展开。

协同反模式

ANTI-PATTERNS · 两种死法

× 反例

把 tool 说明复制进 skill

参数细节在 docstring 与 skill 里**双份漂移**——tool 一改签名，skill 就成了错误说明书。正确写法：**只引用名字 + 判定标准**。

× 反例

假设工具永远可用

不写错误分支——串口断开、504、引脚卡死一来，agent 只能**猜**，猜不出就 47 轮硬试。混沌测试专治这个。

五模式怎么落进一份真实 `SKILL.md`?

→ P09 逐行精读

融合版 Skill 精读

主稿：**firmware-synergy/SKILL.md** · 16 行节选
验证段：**pytest → flash → 读回 → 日志**

firmware-synergy/SKILL.md MARKDOWN · 半成品 · 3 TODO · EXCERPT

```
01 ---
02 name: firmware-synergy
03 description: 编排 devboard MCP 工具执行 写→查→验 硬件在环闭环。          — 1
04   触发词：固件、点灯、调光、开发板、烧录、HIL。
05 ---
06 # 硬件协同工作流（严格按顺序执行）
07 ## 1. 写 — 先查 references/pins.md（LED=GPIO2 · 光敏=ADC34）            — 2
08     firmware_flash 危险不可逆：先获用户同意，MCP 层再校验 confirm
09 ## 2. 查 — ruff check .  # 静态通过 ≠ 能 work，只是第一道关
10 ## 3. 验（硬件在环 · 核心） 返回文本以 OK / ERROR 开头
11     1. 动作：pwm_set（调光）/ gpio_write（开关灯）
12     2. TODO(student)①：读回比对 + 机器可判数值区间                       — 3
13     3. 佐证：get_logs 确认写入事件（防「写进了虚空」）
14 ## 4. TODO(student)②：故障诊断分支（错误文本→处置表）
15 ## 5. TODO(student)③：停止条件（轮次上限·升级报告）
16 # 禁止：未读回比对不得声称「灯亮了」；达上限必须停下报告                  — 4
```

精读四要点

FOUR READINGS · 边注对位

1 引用 tool 的正确写法 L03

只写**名字 + 判定标准**，**不复述参数细节**——细节归 docstring 管。

2 渐进披露支撑判定 L07

引脚与物理特性在 references/pins.md：暗态 ≈800 · 满亮 ≈3000 · 噪声 ±40。

3 3 个 TODO ↔ runner 一一对应 L12–15

补完 skill 再翻译成 runner 同名 TODO——**skill 写不清楚的流程，代码也写不出来。**

4 负面清单堵捷径 L16

不许拿 OK 当「灯亮了」、不许改测试、不许**「再试最后一次」**。

失败升级路径

tool 错误文本 → 诊断分支 → 超限升级给人
这张表 = **TODO② 的参考形状**

DIAGNOSTIC BRANCHES · 症状必须机器可判

症状（TOOL 返回文本）	诊断	处置
ERROR 含 "504" / "timeout" / "超时"	**瞬时故障**	重试，**计 1 轮**；轮次用尽走停止条件
写返回 OK 但读回不一致	**疑似引脚卡死**	board_reset 后重试一次；仍不一致 → 判**不可恢复**，升级
LED 读回正常但 raw 超区间上限	**传感器漂移**	**重试无用——禁止重试**，直接升级
ERROR 不在上表 / 无法归类	**未知**	**不要猜**——直接走停止条件

每条处置都写明**下一步调用哪个 tool、判定标准是什么**——诊断时可加 get_logs 收集板端证据。「无法归类 → 不猜」是分支表最重要的一行。

停止条件

TODO③ · STOP & ESCALATE

ROUND BUDGET

一次 写→验 = 1 轮

上限 ≤ 3 轮

与考官一致：MAX_ROUNDS = 3

1 **升级报告三件套**
诊断结论 + 错误**原文** + 建议动作

2 **禁止重试直接升级**
危险操作未获确认 · 传感器漂移

3 **负面清单兜底**
达上限停下报告，禁**「再试一次」**

判分口径预告：逆风局不要求「必胜」—— recovered 与 escalate 都算对，死循环硬试才算错

实验三：协同改造 + 故障注入

脚手架 **lesson3-synergy-lab/**
模拟器端口 **8766**（与 lesson2 的 8765 错开）

场景 · SCENARIO

「保证它能 work」的**终极检验**：换一个不知道内情的人能**复现闭环**，逆风局（故障注入）下**行为可预期**。

任务 · TASK

把你第 1/2 节课的 skill 升级为**协同版**——填空位而非从零写（防卡壳），再让它经受住**混沌开关**与**跨组验收**。

验收方式 · CROSS-CHECK

机器考官 **make check** 全绿 + B 组只按 A 组的 **SKILL.md** 操作能复现（A 组**禁止口头指导**）。

半成品 BLANKS — **3** 个 **TODO(student)** 空位 · skill 与 runner 一一对应

考官 EXAMINER — **3** 种混沌场景 × **3** 轮上限 **MAX_ROUNDS**

实验材料

SCAFFOLD · 目录树

```
lesson3-synergy-lab/
├── simulator/  lesson2 版 + 混沌开关
├── mcp_server/server.py  毕业版 · 7 tools
├── skills/firmware-synergy/
│   ├── SKILL.md  半成品 · L1 对象
│   3 空位  ① 读回比对  ② 诊断分支  ③ 停止条件
│   └── references/pins.md  判定区间推导依据
├── runner/synergy_runner.py  skill 的机器代理
├── transcripts/  3 份记录 · L0 材料
├── worksheets/transcript-analysis.md
├── tests/test_chaos_hil.py  三场景考官
├── teacher/  answer-key · rubric · 勿看
└── Makefile  setup / chaos-* / check
```

三层任务

L0 保底 → L1 必做 → L2 挑战
28–42′ 动手 · L2 可延续到课后

L0

约 5′
分析 · 人人必做

Transcript 分析 READ THE FAILURES

读 `transcripts/` 三份记录，填工作单：每份的闭环**断在哪一步 / 为什么 / 缺了五模式中哪条** + 5 道快问快答。

不依赖环境——装环境翻车也能拿这 15 分，先做这个。

L1

约 7′
改造 · 必做

协同改造 SYNERGY RETROFIT

补全 `SKILL.md` 的 3 个 `TODO`（读回比对 / 诊断分支 / 停止条件）→ 翻译成 `synergy_runner.py` 同名 3 个 TODO → `make check` **全绿**。

提示 —— 判定区间必须从 **pins.md 物理特性**推导。 **✕ 禁止改 tests / 契约凑通过。**

L2

4′ + 课后
挑战 · 终极检验

故障注入 + 跨组验收 CHAOS TEST

教师 / 对手组开混沌开关，你的组合必须**正确诊断并恢复**，或**按停止条件优雅认输**——**两者都算对**，死循环硬试算错；随后跨组互相注入验收。

TIP · 顺序 先把 skill 写清楚，再翻译成代码 —— **skill 里写不清楚的流程，代码里一定写不出来**。

混沌场景 P13 · 评分 P14 →

三种混沌，三种谎言

教师端 / 对手组注入 · 环境变量 **CHAOS**
随时确认：**curl :8766/state** · 关闭：**make stop**

CHAOS · 1

pin_stuck

写入假成功 · 读回永远旧值

OK? 返回全是 OK
世界没有变

gpio_write 返回 **成功**，gpio_read 电平**永远是旧值**；连 board_reset 也治不好——写与读的世界脱节了。

教师端
make chaos-pin-stuck

正确行为
读回不一致 → board_reset 重试一次 → 仍不一致 → 判「**引脚卡死**」→ **escalate**

CHAOS · 2

serial_timeout

串口抖动 · 随机超时

35% 的请求随机
返回 HTTP 504

每个接口都可能**随机超时**——同一个调用这次 504、下次 OK。故障是**瞬时的**，但重试必须**计轮次**。

教师端
make chaos-serial-timeout

正确行为
识别「504 / timeout」→ **有界重试**（计 1 轮）→ 命中窗口 **recovered**，或轮次耗尽 **escalate**

CHAOS · 3

adc_drift

传感器整体漂移

+300 ADC 读数整体漂移
暗态 ~800 → ~1100

LED 物理耦合**仍生效**，但所有 raw 读数抬高 300——判定区间被整体击穿。**重试无用**：故障不在链路，在传感器。

教师端
make chaos-adc-drift

正确行为
raw 超区间上限 → 判「**传感器漂移**」→ **禁止重试，直接 escalate**

混沌模式下，模拟器的一切「成功返回」都可能是谎言——这正是本实验要你体会的事

怎么评分

100 分制 · L2 尽量**机器考官**判
人工只看诊断结论质量与跨组验收

评分项	验收标准	分值
L0 · Transcript 分析 WORKSHEET	断点填表 **9** + 快问快答 **6**（要点吻合即可）	**15**
L1 · 协同改造 SKILL.MD + RUNNER	编排 **10** · 机器可判标准 **10** · 诊断分支 **10** · 停止条件 **5**；`make check` 全绿拿 25 基本盘	**35**
L2 · 故障注入 CHAOS × 3	三场景各 **10**：`recovered` / `escalate` 都算对	**30**
跨组验收 CROSS-CHECK	复现 **10** · 组合故障 **6** · 验收报告 **4**——别人也能复现	**20**
合计 TOTAL		**100**

红色 20 = 「换人可复现」的直接验收 · 与 L2 合计 **50 分押在逆风局**

L2 判分口径

PER SCENARIO · 10 PTS

GRADING RULE

逆风局要的不是「必胜」，
是行为可预期。

正确诊断恢复、或按停止条件优雅认输——两者都算对。

`recovered` · 灯真亮（防假成功）	**10**
`escalate` · 诊断相符 + 错误原文	**10**
结局对但诊断张冠李戴	**5**
超轮次 / 死循环硬试 / 抛裸异常	**0**
灯没亮却声称成功（假成功）	**0 −5**

✕ 红线（一票否决）：修改 **tests /**、**RunResult** 契约或 **MAX_ROUNDS** 凑通过 = 本实验 **0 分**。

带走的：一个公式，一条闭环

正 → 反 → 合 · 三节课收束
同一份作品，迭代了**三轮**

TAKEAWAY · 核心主张

Vibe coding 的正解 =

写得快 + 验得严

FAST TO WRITE — STRICT TO VERIFY. 而不是「随缘生成」。

1 **Skill = 纪律** 写→查→验 的 SOP · 有限迭代 · 停止条件

2 **MCP = 手** 机器可判的真实反馈 · docstring 即说明书

3 **协同 = 可预期** 编排 · 护栏 · 诊断 · 升级——故障下不崩

「work」的完整定义 = **可交换复现** + **逆风局行为可预期**。

拓展作业

HOMEWORK · 课后

1 **完成 L2 全部场景**
跨组验收做完，交**一页报告**：每场景 轮次 / `status` / 诊断结论。

2 **把方法论带回自己的项目**
为你真实使用的工具链写一份**协同版 skill**：编排 + 机器可判标准 + 诊断分支 + 停止条件。

3 **彩蛋 · 教学自我示范**
把实验指导本身写成一个 skill 发给你的 agent，让 **agent 引导你做实验**——课程本身即示范。

本课程真正想留下的：**一份能证明「它 work」的作品**。

BEYOND · 课程结语

三节课，一份作品 —— skill → 接上硬件 → 协同强化

从今天起，「应该可以了」不算数 —— **拿证据来**：读回比对 · 日志佐证 · 有界重试 · 优雅认输

命令速查：实验三

脚手架 `lesson3-synergy-lab/`
模拟器端口 **8766** · 与 lesson2 的 8765 可并行

命令	作用
`make setup && make doctor`	装依赖（mcp<2 · pytest · ruff）/ 环境自检：venv · SKILL.md 就位 · 模拟器连通性
`make sim`	后台启动模拟器（**无故障注入**，端口 **8766**）
`make chaos-pin-stuck`	CHAOS 1 写入假成功 · **读回永远旧值**（**reset** 也无效 → 应 **escalate**）
`make chaos-serial-timeout`	CHAOS 2 每个接口 **35% 概率随机超时**，返回 **HTTP 504**（瞬时 → 有界重试）
`make chaos-adc-drift`	CHAOS 3 **ADC** 读数**整体漂移 +300**，暗态 **~800 → ~1100**（**LED** 物理耦合仍生效）
`make check`	一键验收：**进程内模拟器 + 三场景 HIL**（无需先 `make sim`）· `MAX_ROUNDS = 3`
`make stop`	停模拟器 / 关闭注入 · 验收完毕必跑
`curl http://127.0.0.1:8766/state`	随时确认当前注入状态（**chaos 字段**）· 也是防「假成功」的旁路证据

真人 AGENT 版注册

Claude Code `claude mcp add devboard -e SIMULATOR_URL=http://127.0.0.1:8766 -- .venv/bin/python mcp_server/server.py`

OpenCode opencode.json 的 mcp 段：command = .venv/bin/python + server.py，environment 带 SIMULATOR_URL · **skill 安装** 复制 firmware-synergy/ 到 .claude/skills/ 或对应目录（全表见第一课 P14）